

Java exception handling

How exceptions work, how to structure handling, and the traps. The tone here is how I would explain each one to a teammate sitting next to me, so it is plain and spoken; the scenario questions are fully how I would talk them through.

1. What is an exception, why does Java use them, and what is the point of handling them?

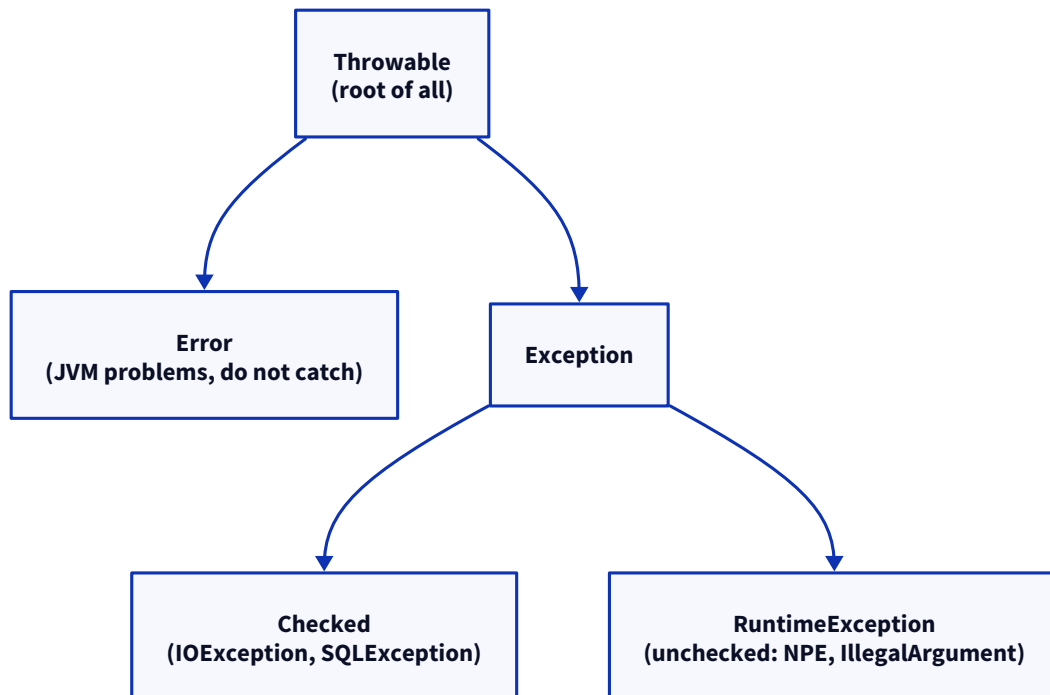
So an exception is just Java's way of saying **something went wrong at runtime that the normal flow cannot deal with**: a file is missing, a number will not parse, you dereferenced null. Instead of returning a magic error code that everyone forgets to check, Java **throws an object up the call stack** until someone catches it. That is why Java uses them, they separate the "what to do when it fails" from the happy path, so your main logic stays readable. And the point of handling them is to **recover or fail cleanly**: retry, fall back, log with context, or return a sensible error, rather than letting the program crash with a raw stack trace in the user's face.

2. Error v/s Exception, and why don't we catch Errors?

Here is the split. Both extend `Throwable`, but **Exception** is for problems your program can reasonably deal with (a bad input, a missing file), and **Error** is for problems the JVM itself is in trouble with (`OutOfMemoryError`, `StackOverflowError`). The reason we generally **do not catch Errors** is that there is usually nothing useful you can do, if the heap is exhausted, catching it and continuing just gets you another one a moment later. So the rule of thumb I give people: handle Exceptions, let Errors bring the process down.

3. Explain the Java exception hierarchy.

Everything hangs off **Throwable**, which is the root, the only thing you can `throw` or `catch`. Under it are two branches: **Error** (JVM-level, leave it alone) and **Exception** (your zone). Inside **Exception**, the important fork is **RuntimeException and its subclasses, which are unchecked**, versus **everything else under Exception, which is checked** (like `IOException`, `SQLException`). So when someone asks "what is `RuntimeException`", it is the base of the unchecked family (`NPE`, `IllegalArgumentException`, and friends).



4. Checked v/s unchecked exceptions.

The difference is really about the compiler. **Checked exceptions must be either caught or declared** with `throws`, the compiler forces you to acknowledge them, and they are meant for **recoverable, expected** failures (a file might not be there). **Unchecked exceptions (RuntimeException and its children) are not enforced** by the compiler, and they usually signal a **programming bug** (you passed null, you indexed past the end).

- Common **checked**: `IOException`, `SQLException`, `FileNotFoundException`.
- Common **unchecked**: `NullPointerException`, `IllegalArgumentException`, `IllegalStateException`, `IndexOutOfBoundsException`, `NumberFormatException`, `ClassCastException`, `ArithmeticException`.

5. What happens if an exception is not handled, and can an app recover from every exception?

If nobody catches it, it **propagates up the call stack**, and if it reaches the top (the thread's entry point) the **thread dies** and Java prints the stack trace; on the main thread that ends the program. And no, you **cannot recover from every exception**, and you should not try. Some are recoverable (retry a flaky network call), some are not (a bug that left your state inconsistent), and Errors usually are not at all. Part of the skill is knowing which is which and not blindly catching everything.

6. Why does Java have checked exceptions, and what are their pros and cons?

Java's idea was to **make failure part of the method's contract**, so a caller cannot forget that a call might fail, the compiler nags you. That is the **advantage**: it forces you to think about the failure path, and the signature documents what can go wrong. The **downside**, and the reason they are controversial, is **boilerplate and leaky abstractions**: they clutter code with try-catch or `throws`, they do not compose with lambdas and streams,

and people cheat by catching and swallowing them or wrapping everything in a `RuntimeException`. So the feature meant to improve error handling often makes it worse in practice.

7. When do you pick checked v/s unchecked, and why do Spring and modern frameworks prefer unchecked?

The way I decide: use a **checked** exception when the caller can **realistically recover** and you want to force them to handle it; use an **unchecked** exception for **programming errors and for failures the caller usually cannot do anything about**. In modern code the lean is heavily toward **unchecked**, and that is why **Spring converts checked exceptions (like `SQLException`) into unchecked `DataAccessException`s**, it does not want every service method up the stack polluted with `throws SQLException` for something they cannot fix anyway. On **business validation failures**, I keep those unchecked too (an `IllegalArgumentException` or a custom unchecked domain exception), because littering the codebase with checked validation exceptions buys you nothing. The theme: checked only when handling is both possible and expected.

8. How do checked exceptions affect API design, and why is `throws Exception` bad?

Checked exceptions **become part of your API's signature**, so once a public method declares `throws SomethingChecked`, every caller is bound to it, and adding a new checked exception later **breaks callers**. That is why you design them deliberately. And `throws Exception` is **poor design** because it is the vaguest possible promise: it tells the caller "this might fail somehow" without saying how, forces them to catch the broad `Exception` (which then hides real problems), and defeats the whole point of typed exceptions. Declare the specific exceptions you actually throw, not the top of the tree.

9. If unchecked exceptions are more convenient, why do checked exceptions still exist?

Because for a **genuinely recoverable, expected** failure, the compiler forcing you to handle it is actually valuable, you cannot accidentally ignore it. The classic good case is something like a file or network operation where the caller really should have a plan B. So they still exist for that "you must deal with this" contract; the problem is only that they got overused for things that were never recoverable.

10. How does try-catch-finally work, and what is the execution order?

Plain version: the `try` holds the risky code, a matching `catch` runs if that type of exception is thrown, and `finally` runs afterwards no matter what, for cleanup. Order is: try runs until something throws, control jumps to the first matching catch, and then finally runs at the end (whether or not an exception happened, and even if the catch itself throws). On the structural rules people ask about:

- A `try` **can skip catch if it has a finally** (try-finally is legal).
- A `try` **can skip finally if it has a catch**.
- A `finally` **can exist without a catch** (that is try-finally).
- What you cannot have is a bare `try` with neither.

11. Is `finally` always executed, and what about returning or throwing inside it?

Almost always, but not literally always. **`finally` does not run** if the JVM stops first, `System.exit()`, a JVM crash, or the thread being killed, and obviously not if you never enter the try. The gotchas: if an **exception is thrown inside finally**, it **replaces** whatever was propagating from the try, so you silently lose the original, which is nasty. And **`return` inside finally** is legal but a bad idea, because it **overrides any return or exception from**

the try, so a real result or a real error just vanishes. My rule: keep finally for cleanup only, never put return or throw in it.

12. Difference between `throw` and `throws`, and when do you use each?

Easy to mix up by name. `throw` is the statement that actually **raises an exception** right now (`throw new IllegalArgumentException("id is null")`). `throws` is in the **method signature** and just **declares** that the method might throw certain checked exceptions, so callers know. You can **declare multiple** with `throws A, B`, and a method can **throw both checked and unchecked** (you only have to declare the checked ones). So: `throw` does it, `throws` warns about it.

13. Exception propagation, rethrowing, wrapping, and preserving the root cause.

Propagation is just the exception bubbling up the call stack until someone catches it. **Rethrowing** is catching it, maybe doing something (logging), and throwing it onward. **Wrapping** is catching a low-level exception and throwing a more meaningful one (`catch SQLException, throw OrderNotFoundException`). The one thing you must not forget when wrapping is to **preserve the root cause**: pass the original in as the cause (`throw new OrderException("...", e)`), so the **original stack trace survives** in the "Caused by:" chain. Lose that, and you have thrown away the one thing that tells you where it really broke.

14. When and how do you create a custom exception, and should it extend `Exception` or `RuntimeException` ?

I create a custom exception when I want a **domain-meaningful failure** that callers can catch specifically, `InsufficientFundsException` says far more than a generic `IllegalStateException`. You make one by **extending `Exception` (checked) or `RuntimeException` (unchecked)** and adding constructors. On which to extend: I default to **`RuntimeException` (unchecked)** unless the caller genuinely must handle it, which matches how modern frameworks lean. So most custom domain exceptions I write are unchecked.

15. What should a custom exception contain, and what is the exception translation pattern?

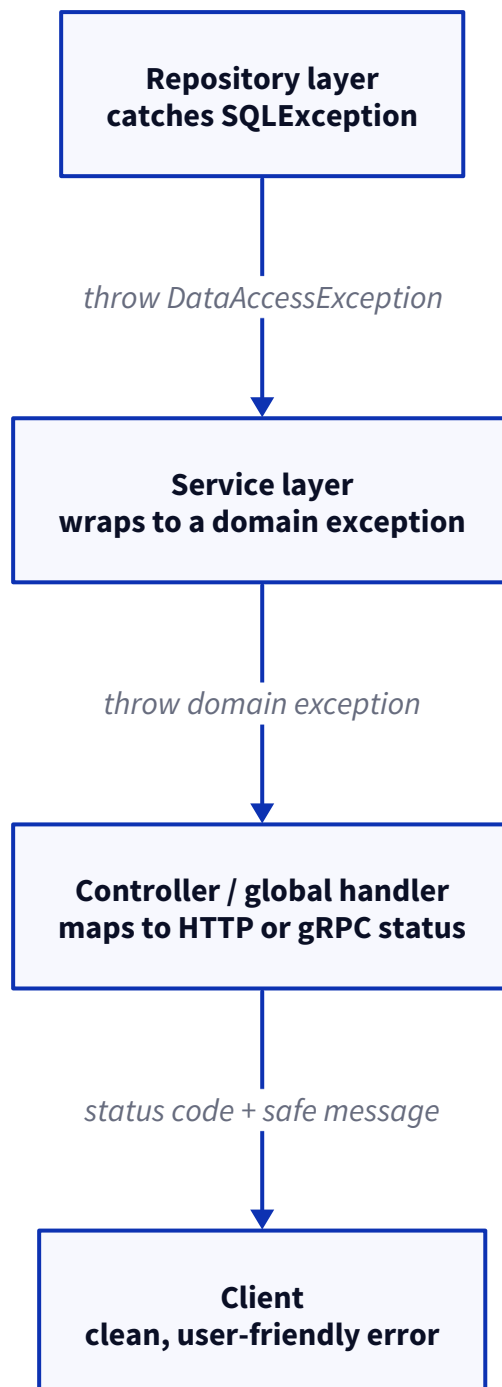
A good custom exception carries **enough context to diagnose the failure without guessing**: a clear message, the **original cause** (constructor that takes a `Throwable`), and often an **error code** and **relevant context** (the entity id, the operation) so logs and API responses can use it. Give it the standard **constructors** (message, message + cause). I like **domain-specific** exceptions at the business layer and sometimes **layer-specific** ones. The **exception translation pattern** ties into Q13's wrapping: as an exception moves up, each layer **translates it into its own vocabulary**, the repository's `SQLException` becomes the service's `DataAccessException` becomes a clean API error, so higher layers are not coupled to lower-layer details. On naming, end them in `Exception` and make the name say what failed.

16. Are exceptions expensive, and why?

Yes, more than people expect, and the cost is almost entirely in **creating the exception, not catching it**. When you construct one, the JVM **captures the whole stack trace** (`fillInStackTrace`), walking every frame, and that is the slow part. So a try-catch that never fires is basically free, but **throwing exceptions in a hot loop** (using them for normal control flow) will hurt. That is the real reason for the "don't use exceptions for control flow" rule, it is a performance point as much as a style one.

17. How should exceptions be handled across layers, and in REST and gRPC?

The pattern I follow is **translate as you go up, and handle centrally at the top**. The **repository layer** catches the raw data exceptions (`SQLException`) and wraps them into domain terms. The **service layer** works in domain exceptions and does not leak persistence details. The **controller layer** does not try-catch everything itself; instead a **global exception handler** (in Spring, `@ControllerAdvice`) maps each exception type to a clean response. For **REST** that means an exception maps to the right **HTTP status** (404 for not-found, 400 for bad input, 500 for the unexpected) plus a safe JSON body. For **gRPC** it maps to the right **status code** (`NOT_FOUND` , `INVALID_ARGUMENT`) instead. The point of the global handler is you get **consistent, user-friendly errors** in one place, and you never expose internal stack traces to the caller.



18. What is try-with-resources, and why is it preferred over finally?

Try-with-resources lets you **declare a resource in the `try(...)`** and **Java closes it automatically** when the block exits, success or failure. It was introduced because the old **try-finally cleanup was verbose and easy to get wrong**, people forgot to close, or their close in finally threw and masked the real exception. So it is

preferred because it is **shorter, cannot forget to close, and handles the close-throwing case correctly** (see suppressed exceptions next). Any time I open a file, a stream, or a connection, this is the form I use.

19. AutoCloseable v/s Closeable, closing order, and suppressed exceptions.

To be usable in try-with-resources, a class implements **AutoCloseable**, whose `close()` can throw any **Exception**. **Closeable** is the older interface (from I/O) that **extends AutoCloseable** but narrows `close()` to throw **IOException**, so **Closeable** is really a specialised **AutoCloseable**. If you declare several resources, they are **closed in reverse order** of how you opened them (last opened, first closed), like a stack. Now the neat part: **if the try body throws AND `close()` also throws**, Java keeps the body's exception as the main one and attaches the close exception as a **suppressed exception**, so you do not lose either. You read them off with **`Throwable.getSuppressed()`**. That is exactly the case old try-finally got wrong, the close exception used to clobber the real one.

20. A database becomes unavailable. Which exception strategy would you use?

The way I would handle it: treat it as a **transient, recoverable** failure, so I would **retry with backoff** a couple of times in case it is a blip, and wrap it in a **circuit breaker** so that if the database is genuinely down I fail fast instead of piling up doomed calls. The exception itself I would **catch low and translate** into a clear service-level exception, and at the edge return a **503-style "temporarily unavailable"**, not a raw **SQLException**. What I would not do is swallow it or retry forever. So: retry the blip, break on the outage, degrade gracefully, and never leak the raw exception.

21. An external API returns invalid data. How would you handle it?

I would **validate at the boundary and fail fast there**. The moment the response comes back, I check it, and if it is malformed I throw a **specific exception** (something like **`InvalidResponseException`**) right at the integration layer, rather than letting bad data flow deeper and blow up somewhere confusing later. I would **log the offending payload** (carefully, no secrets) so we can see what they actually sent, and decide per case whether to **fall back** to a default or **reject** the operation. The principle is to keep bad external data from contaminating my domain, catch it at the door.

22. A file upload fails halfway through processing. What should happen?

The key word is **cleanup and consistency**. If it fails partway, I need to make sure I do not leave a **half-written file or a half-updated record**, so I would process in a way that is **atomic or rollback-able**: write to a temp location and only commit on success, and wrap the database side in a **transaction** so a failure rolls it back. Resources (the stream) get closed via **try-with-resources**. And I would return a clear error telling the user it failed so they can retry, ideally **idempotently** so a retry does not create a duplicate. So: do not leave partial state, clean up, and make the retry safe.

23. How would you design custom exceptions for a payment processing system?

For payments I would build a **small hierarchy of domain exceptions** under one base like **`PaymentException`**, with specific ones for the real cases: **`InsufficientFundsException`**, **`PaymentDeclinedException`**, **`PaymentTimeoutException`**, **`DuplicatePaymentException`**. Each carries an **error code and context** (transaction id, provider) for logs and support. The important design call is **which are retryable**: a timeout might be retried (with idempotency), a decline must not be. I would keep them **unchecked** but catch them explicitly at the boundary and map each to a clear status and a safe message, never exposing the provider's raw error. *(Adapt to a system you have built.)*

24. Describe a production incident where exception handling helped find the root cause.

The pattern that has saved me more than once is **preserving the cause chain**. In one case an operation was failing intermittently, and because the exceptions were **wrapped with the original cause attached**, the "Caused by:" chain in the logs pointed straight past our service-level `OrderException` down to the real culprit, **a timeout on a downstream call**, so we did not have to guess. The lesson I took from it is that good exception handling is really about **diagnosability**: wrap with meaning but always keep the root cause and log with enough context. The times it has hurt were the opposite, a swallowed exception or a lost stack trace that turned a five-minute fix into an afternoon. *(Adapt to a real incident you owned.)*

25. How would you design an exception-handling strategy for a large microservices platform?

I would make it **consistent and centralised across services**. That means a **shared error model**: every service maps exceptions to the same shape (an error code, a message, a correlation id) so callers and dashboards see one format. Each service uses a **global handler** (Spring `@ControllerAdvice` or the gRPC equivalent) so mapping lives in one place, not scattered. I would **propagate a correlation/trace id** through every call so an exception in one service can be traced back across hops. Transient cross-service failures get **retry, timeout, and circuit breaker** (the resilience patterns), and nothing ever returns a raw stack trace over the wire. The goal is that a failure looks the same and is traceable no matter which service threw it. *(Adapt to your platform.)*

26. Why shouldn't you catch `NullPointerException` in most cases?

Because an NPE is almost always a **bug, not a condition to recover from**, it means you dereferenced something that should not have been null, so catching it just **hides the bug** and lets the program limp on in a broken state. The right move is to **prevent it** (validate inputs, use `Optional`, `Objects.requireNonNull` at boundaries), not to catch it after the fact. Catching NPE is treating the symptom and burying the cause.

27. Exception handling best practices.

- **Catch the most specific exception**, not `Exception` or `Throwable`, so you do not accidentally swallow things you did not mean to.
- **Never swallow an exception** with an empty catch block; at minimum log it with context.
- **Log correctly**: log it once, at the level that will actually handle it, with the cause, and do not log-and-throw at every layer.
- **Preserve the original exception** as the cause when you wrap.
- **Do not use exceptions for normal control flow**, it is slow and it reads badly.
- **Write meaningful messages** (what failed and the key context), not "error occurred".
- **Fail fast** on programming errors rather than failing silently.
- **Centralise handling** at the boundary with a global handler.

28. Common exceptions.

Exception	When it shows up
<code>NullPointerException</code>	Using a reference that is null
<code>IllegalArgumentException</code>	A method got an invalid argument
<code>IllegalStateException</code>	An object is in the wrong state for the call
<code>IndexOutOfBoundsException</code>	Index past the end of a list or array
<code>NumberFormatException</code>	Parsing a non-numeric string to a number
<code>ClassCastException</code>	Casting to a type the object is not
<code>ArithmeticException</code>	Illegal maths, like divide by zero
<code>IOException</code>	An I/O operation failed (checked)
<code>SQLException</code>	A database access error (checked)
<code>FileNotFoundException</code>	The named file does not exist (checked)
<code>NoSuchElementException</code>	Reading past the end of an iterator
<code>UnsupportedOperationException</code>	Calling an unsupported method (e.g. modifying an immutable list)

29. Common mistakes.

- **Catching generic `Exception`** (or `Throwable`), which hides problems you never meant to handle.
- **Swallowing exceptions** in an empty catch block, the single worst habit.
- **Logging and rethrowing the same exception** at every layer, so the logs are full of duplicates.
- **Throwing generic `Exception`** instead of a specific type.
- **Returning from a `finally` block**, which silently overrides the real result or error.
- **Losing the original stack trace** by wrapping without passing the cause.
- **Using exceptions for input validation** or normal control flow.
- **Ignoring suppressed exceptions** from try-with-resources.
- **Exposing internal exception messages** (or stack traces) to end users.
- **Creating too many custom exception types**, so nobody knows which to throw.